

Prática #04 - 2025.1

Objetivos

Fazer, finalmente, o teste do multiplicador de 4 bits da primeira prática e integrar a comunicação $PC \Leftrightarrow HPS \Leftrightarrow FPGA$.

Instruções Gerais

- Como entrega para esta atividade:
 - mostrar o sistema funcionando em sala;
 - colocar no classrrom como resposta o link para um repositório no seu github com o projeto feito (contendo o sistema - PC, HPS e FPGA).

1 O Sistema

Nosso sistema de testes deve ser configurado como mostrado no diagrama de blocos da Figura 1:

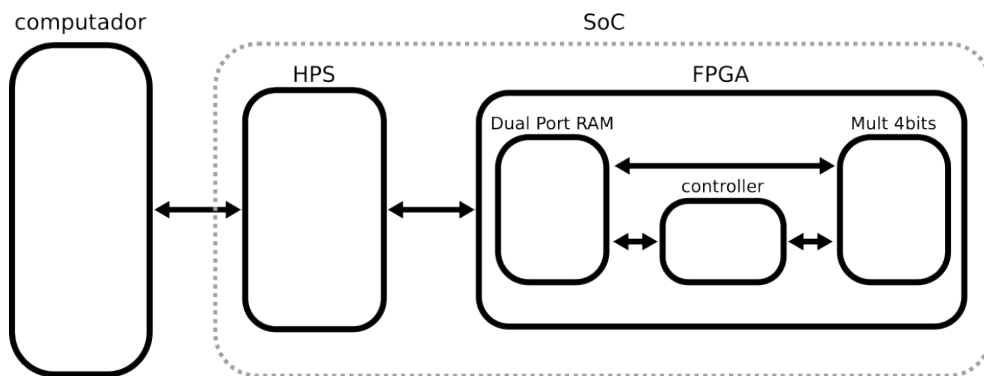


Figura 1: Diagrama de blocos do sistema de testes.

- **Computador:** *script* Python responsável por gerar as entradas do sistema, enviá-las para o **SoC**, receber as respostas enviadas pelo **SoC** e verificá-las;
- **SoC:**
 - **HPS:** programa em C responsável por receber as entradas vindo do **Computador**, tratá-las e alimentar a **Dual Port RAM**. Controlar o início e verificar o fim do cálculo feito pelo **Mult 4bits**. Após o término do cálculo, enviar a resposta para o **Computador**;

– **FPGA:**

- * **Dual Port RAM:** um IP core da Altera, ela possui quatro palavras de 32 bits cada. Se achar pertinente, pode reduzir o tamanho do barramento para 8 bits, por exemplo. Uma sugestão de função relacionada a cada endereço é dada na Tabela 1.
- * **Mult 4bits:** módulo criado na Prática 1;
- * **controller:** módulo a ser criado responsável por
 1. ler a **Dual Port RAM** no endereço *control* para saber quando fazer uma nova operação;
 2. ler a **Dual Port RAM** no endereço *data in* os operandos A e B e alimentar o **Mult 4bits**;
 3. gerar o sinal de *enable* para inicializar o cálculo;
 4. gravar o valor de Y na **Dual Port RAM** no endereço *data out* uma vez que o cálculo tiver terminado;
 5. gravar na **Dual Port RAM** no endereço *status* indicando que o cálculo foi terminado.

Tabela 1: Sugestão de função das palavras guardadas na **Dual Port RAM**.

Endereço	Nome	Funções
00	<i>control</i>	bit[0] = iniciar cálculo
01	<i>data in</i>	bit[3:0] = A; bit[7:4] = B
10	<i>data out</i>	bit[7:0] = Y
11	<i>status</i>	bit[0] = fim do cálculo

2 Barramento Avalon-Memory-Mapped

Para se comunicar com a **Dual Port RAM** estamos utilizando um barramento proprietário da Altera/Intel, o Avalon Memory-Mapped (Av-MM) [1]. Embora possa ter diferentes configurações para dispositivos *master* ou *slave*, vamos apresentar apenas os sinais para configuração *slave* necessários neste projeto. Os sinais do barramento são sumarizados na Tabela 2. Embora pelas configurações do barramento ele possa ter um comprimento de até 1024 bits, na prática, o **Lightweight HPS-to-FPGA Bridge** suporta até 32 bits apenas [2], a escolha de um valor maior que 32 vai necessitar de mais de uma escrita ou leitura por comando.

Tabela 2: Sinais do barramento Av-MM *slave* [1].

Nome	Comprimento (bits)	Direção	Descrição
clk	1	in	sinal de sincronismo
chipselct	1	in	ignora o barramento caso <code>chipselct = 0</code>
address	1-32	in	endereço de escrita ou leitura
readdata	1-1024	out	sinal de leitura
write	1	in	escreve <code>writedata</code> no <code>address</code>
writedata	2^x $x \in \{3, 4, \dots, 10\}$	in	sinal de escrita
byteenable	2^x $x \in \{1, 2, \dots, 7\}$	in	byte dentro da palavra

2.1 Leitura

Da forma em que configuramos o barramento da **Dual Port RAM**, a leitura é feita de forma assíncrona: basta deixar o valor de **address** estável por um ciclo de **clock** e ler o sinal **readdata**.

2.2 Escrita

A escrita é síncrona: primeiro estabiliza-se os valores de **writedata**, **address** e **byteenable**, então mantém o sinal **write** em nível lógico alto até a próxima borda de subida do **clock**, quando então o valor de **writedata** é gravado no endereço selecionado.

Referências

- [1] Altera, “Avalon memory-mapped interface - specification.” online, 2006. Disponível em: https://www.cs.columbia.edu/~sedwards/classes/2007/4840/mnl_avalon_spec.pdf.
- [2] Terasic, “De10-nano user manual.” Terasic Inc, 2023.